PROJECT DOCUMENTATION

LINUX SIGNAL-BASED PROCESS CONTROLLER

A command-line approach to controlling Linux processes using operating-system signals

SUBMITTED BY

K. Shashank Reddy	2520030215
Ch. Charan Sai	2520030321
K. Yagnesh	2520030121

Operating Systems / Linux • Hackathon Project

Abstract

The Linux Signal-Based Process Controller is a proposed command-line utility for managing selected Linux processes through operating-system signals. Its intended operations are pause, resume, terminate, and restart. The user supplies an action and a process identifier (PID) or process name. The controller validates the target, requests the relevant operation, and reports the outcome.

The project demonstrates core operating-systems concepts, including process identification, signals, process states, permissions, and process creation. The implementation can use Linux interfaces such as `kill()`, `fork()`, `exec()`, `waitpid()`, and information exposed through `/proc`, depending on the final implementation.

1. Introduction

A Linux system may run many processes at the same time. Users and developers sometimes need to temporarily stop a process, continue it later, request its termination, or relaunch a program. These actions can be performed manually through terminal tools, but repeated manual commands can be inconvenient and may lead to mistakes when identifying the correct process.

This project aims to bring common process-control actions together behind a simple interface. It is also a practical exercise in understanding how a user-level program communicates with the Linux operating system.

2. Problem Statement

Process management commonly requires the user to identify a process, remember the appropriate command or signal, and verify the resulting state. Restarting a process can require additional knowledge of the original executable and its arguments.

The project proposes a single utility that accepts a requested action and a target process, validates the target, performs the supported operation, and displays a useful status message.

3. Objectives

- Provide a simple command-line interface for process-control actions.
- Support pausing a process using SIGSTOP.
- Support resuming a stopped process using SIGCONT.
- Request process termination using SIGTERM, with SIGKILL reserved for a necessary forced stop.
- Identify invalid targets and report permission or operation errors clearly.
- Explore restarting a process when its original executable and arguments can be recovered safely.
- Demonstrate Linux process states and signal handling with a harmless test process.

4. Scope of the Project

The intended scope is Linux user-level process management. The controller may accept a PID or process name and provide terminal-based feedback. The exact supported commands depend on the implementation completed by the team.

The tool should be used only for processes the user owns or is authorized to manage. System-critical processes should not be targeted during demonstrations. Process-name matching must be handled carefully because a name can match more than one process.

5. System Design

The proposed system follows a straightforward processing pipeline:

- **Command-line input:** Read the requested action and target PID or process name.
- **Process lookup:** Locate the target and check whether it exists.
- **Validation:** Check input format and handle permissions or unavailable targets.
- **Signal dispatcher:** Map the requested action to the appropriate Linux signal.
- **Operating-system interaction:** Ask Linux to deliver the signal or carry out the supported restart flow.
- **Status output:** Report success or explain the error to the user.

6. Linux Signals Used

Linux signals notify a process that an event or request has occurred. The following signals are relevant to the proposed controller:

Signal	Purpose	Important detail
`SIGSTOP`	Pause/stop a process	Cannot be caught or ignored by the target process.
`SIGCONT`	Continue a stopped process	Allows execution to continue.
`SIGTERM`	Request termination	The process may handle or block it; exit is not guaranteed immediately.
`SIGKILL`	Force termination	Cannot be caught or ignored; use only when justified.

7. Implementation Approach

A typical implementation can follow these steps. Adapt the description to match the code actually used by the team.

- **Step 1 — Parse input:** Read the action and process identifier or name.
- **Step 2 — Resolve target:** Locate the process and validate that the target is appropriate.
- **Step 3 — Perform action:** Send the signal that corresponds to pause, resume, or termination.
- **Step 4 — Handle errors:** Check system-call return values and report conditions such as an invalid PID or insufficient permission.

- **Step 5 — Restart:** If implemented, capture the executable and arguments, terminate the old process, wait for it to exit, and launch a new instance. Restart cannot be guaranteed for every arbitrary process.

Possible Linux interfaces

`kill()` sends a signal to a process; `fork()` creates a child process; `exec()` replaces a process image with a program; `waitpid()` waits for a child process state change; and `/proc` exposes process information. Include only interfaces actually used in the final code.

8. Technologies and Tools

- **Operating system:** Linux / Ubuntu.
- **Programming language:** C or Python, according to the final implementation.
- **Operating-system concepts:** processes, PIDs, signals, permissions, process states, and process creation.
- **Development and verification tools:** GCC, Bash, Git, `ps`, and `top`, where applicable.

9. Testing Plan

Use a harmless test process such as `sleep 300`. Record actual command output and observed process states before reporting a test as passed.

Test	Action	Verification
Pause	Send SIGSTOP	Confirm process state indicates stopped (often T in ps output).
Resume	Send SIGCONT	Confirm the process continues running.
Terminate	Send SIGTERM	Confirm process exits, if it accepts the request.
Invalid PID	Enter a non-existent PID	Show a clear error and do not affect another process.
Restart	Use implemented restart operation	Verify the relaunched process; PID may change.

Reporting rule: These are test cases and expected observations, not confirmed results. Replace them with actual outcomes after running the program.

10. Expected Results

- Pause: the selected process becomes stopped when SIGSTOP is successfully delivered.
- Resume: the selected stopped process is allowed to continue after SIGCONT.
- Terminate: the process receives a termination request; actual exit must be checked.
- Restart: where implemented correctly, a new process instance is launched from the recovered command.
- Error handling: invalid targets and permission problems produce informative messages.

11. Limitations and Safety

- A user may not have permission to control every process.
- A process name may match multiple processes; ambiguous matches must be handled carefully.
- SIGTERM requests a clean exit but does not guarantee immediate termination.
- SIGKILL prevents cleanup and should be used only when necessary.
- Restarting an arbitrary process is difficult if its executable path, arguments, environment, or working directory cannot be recovered.
- Test with non-critical processes and never use the demo to disrupt system services.

12. Future Enhancements

- Maintain an action log with timestamps and process identifiers.
- Add a confirmation prompt before termination or forced termination.
- Support scheduled process actions.
- Add process monitoring and optional crash-restart functionality.
- Create a graphical dashboard while retaining the command-line interface.

13. Conclusion

The Linux Signal-Based Process Controller is designed to bring common process-management operations into one command-line workflow. It provides a practical way to study Linux signals, process identification, permissions, and process states. The quality of the final system should be demonstrated through actual tests, clear error handling, and safe operation on authorized processes.

References

- Linux manual pages: `man 7 signal`.
- Linux manual pages: `man 2 kill`, `man 2 fork`, `man 3 exec`, and `man 2 waitpid`.
- Linux process information: `man 5 proc`.

Note: This document describes the project design and a suggested test plan. Confirm that the programming language, functions, restart behavior, and results match the team's actual implementation before submission.